Code

Brain MRI Image Segmentation Using K-Means Clustering

03 — Model Selection and Clustering Stability

Keman Xiang | PhD Application Portfolio

- 1 Project Overview
  - 1.1 Background and motivation
  - 1.2 Research questions
  - 1.3 Analysis roadmap
  - 1.4 Data provenance and scope
- 2 1. Data Import and Integrity Checks
  - 2.1 1.1 Locate and read the original CSV files
  - 2.2 1.2 Validate the matrices and pixel vectors
- 3 2. K-Means Model-Selection Framework
  - 3.1 2.1 What we optimize
  - 3.2 2.2 Estimation Strategy
- 4 3. Elbow Method and Marginal Improvements
  - 4.1 3.1 Visualize the WCSS curve
  - 4.2 3.2 Quantify gains from each additional cluster
- 5 4. Compare Segmented Images and Cluster Centroids
  - 5.1 4.1 Preserve image orientation
  - 5.2 4.2 Compare k = 3, 4, 5, 6, and 8
  - 5.3 4.3 Inspect centroid positions and pixel proportions
- 6 5. Sensitivity to Random Initialization
  - 6.1 5.1 Why initialization matters
- 7 6. Model Selection and Interpretation
  - 7.1 6.1 Synthesize fit, complexity, and stability
  - 7.2 6.2 Important limitations
- 8 7. Conclusion
- 9 8. Reproducibility
- 10 References

1 Project Overview

Project: Brain MRI Image Segmentation Using K-Means Clustering
Module: 03 — Model Selection and Clustering Stability
Environment: R / RStudio / R Markdown
Training image: `healthy.csv`
Companion image: `tumor.csv` (validated but not used to choose *k*)

1.1 Background and motivation

We investigate how the number of K-Means clusters influences grayscale brain MRI segmentation. Each pixel is represented by one numeric intensity between zero and one. We group pixels by intensity, rather than by their physical location or verified anatomical identity. A small number of clusters compresses the intensity scale into broad regions; a larger number preserves finer intensity distinctions but increases segmentation complexity.

We compare candidate cluster counts using within-cluster sum of squares (WCSS), relative improvements in fit, segmented-image comparisons, and repeated random initializations. We use the five-cluster segmentation as a reference point while assessing whether nearby alternatives are reasonable. We do **not** infer clinical tumor boundaries or diagnostic accuracy from these unsupervised labels.

1.2 Research questions

1. How does total within-cluster variation change as we increase the number of clusters from two to ten?
2. At which values of *k* do additional clusters produce diminishing reductions in WCSS?
3. How do the segmentation patterns and intensity centroids differ across candidate models?
4. How sensitive are the fitted solutions to K-Means initialization, and what does that imply for our model choice?

1.3 Analysis roadmap

Source-data validation → K-Means objective → WCSS sweep (k = 2–10) → Marginal improvements and elbow inspection → Segmentation comparison → Initialization stability → Selection and limitations.

We treat model selection as a balance between intensity fidelity, parsimony, and stability. We do not treat the elbow as a statistically proven optimum.

1.4 Data provenance and scope

Source	Role in our analysis
`healthy.csv`	Fit and compare all K-Means models using these observed grayscale pixels.
`tumor.csv`	Verify the companion image's format without fitting, validating, or selecting *k* on it.
`Unit6_Recitation.pdf`	The image-segmentation and clustering background.
`Unit6_Recitation.R`	Matrix-to-vector representation and five-cluster K-Means segmentation workflow.
`week6_3.2.R`	WCSS / scree-plot strategy across *k* = 2–10 and examine the four- and five-cluster alternatives.

Unit of analysis. We observe pixels, not independent patients. We have no tissue annotations, segmentation masks, acquisition metadata, or clinical outcomes. We therefore evaluate numerical clustering properties, not medical segmentation accuracy.

2 1. Data Import and Integrity Checks

2.1 1.1 Locate and read the original CSV files

Read the data.

Hide

```
find_data_dir <- function() {
  candidates <- c("data", "../data", ".", "..")
  found <- vapply(candidates, function(path) {
    file.exists(file.path(path, "healthy.csv")) &&
      file.exists(file.path(path, "tumor.csv"))
  }, logical(1))
  if (!any(found)) {
    stop("Cannot find healthy.csv and tumor.csv. Place both in data/.")
  }
  normalizePath(candidates[which(found)[1]], mustWork = TRUE)
}

data_dir <- find_data_dir()
healthy_df <- read.csv(file.path(data_dir, "healthy.csv"),
                       header = FALSE, check.names = FALSE)
tumor_df <- read.csv(file.path(data_dir, "tumor.csv"),
                     header = FALSE, check.names = FALSE)
healthy_matrix <- as.matrix(healthy_df)
tumor_matrix <- as.matrix(tumor_df)
```

Code explanation. We use `header = FALSE` because each CSV cell is an image intensity, not a column heading. We convert the imported data frames into numeric matrices to preserve the original two-dimensional image shapes.

2.2 1.2 Validate the matrices and pixel vectors

We verify that both images contain finite numeric intensities within the expected grayscale interval. We then flatten the matrices using R's column-major ordering. We compare the reconstructed pixel values without requiring the CSV-derived matrix names to match.

Hide

```
validate_image <- function(x, image_name) {
  stopifnot(is.matrix(x), is.numeric(x), length(dim(x)) == 2L,
            nrow(x) > 0L, ncol(x) > 0L)
  if (anyNA(x) || any(!is.finite(x))) {
    stop(image_name, " contains missing or non-finite pixel values.")
  }
  if (any(x < 0 | x > 1)) {
    stop(image_name, " has intensities outside [0, 1].")
  }
  invisible(TRUE)
}
validate_image(healthy_matrix, "Healthy MRI")
validate_image(tumor_matrix, "Tumor MRI")

healthy_vector <- as.vector(healthy_matrix)
tumor_vector <- as.vector(tumor_matrix)

healthy_check <- matrix(healthy_vector, nrow = nrow(healthy_matrix),
                        ncol = ncol(healthy_matrix))
tumor_check <- matrix(tumor_vector, nrow = nrow(tumor_matrix),
                      ncol = ncol(tumor_matrix))
stopifnot(
  length(healthy_vector) == prod(dim(healthy_matrix)),
  length(tumor_vector) == prod(dim(tumor_matrix)),
  identical(as.vector(healthy_check), healthy_vector),
  identical(as.vector(tumor_check), tumor_vector),
  isTRUE(all.equal(unname(healthy_check), unname(healthy_matrix))),
  isTRUE(all.equal(unname(tumor_check), unname(tumor_matrix)))
)

input_summary <- data.frame(
  Image = c("Healthy MRI", "Tumor MRI"),
  Rows = c(nrow(healthy_matrix), nrow(tumor_matrix)),
  Columns = c(ncol(healthy_matrix), ncol(tumor_matrix)),
  Pixels = c(length(healthy_vector), length(tumor_vector)),
  Minimum = c(min(healthy_vector), min(tumor_vector)),
  Maximum = c(max(healthy_vector), max(tumor_vector)),
  Missing = c(sum(is.na(healthy_vector)), sum(is.na(tumor_vector)))
)
knitr::kable(input_summary, digits = 4,
             caption = "Verified dimensions and intensity ranges of the s
ource images")
```

Verified dimensions and intensity ranges of the source images

Image	Rows	Columns	Pixels	Minimum	Maximum	Missing
Healthy MRI	566	646	365636	0	1	0

Image	Rows	Columns	Pixels	Minimum	Maximum	Missing
Tumor MRI	571	512	292352	0	1	0

Output interpretation. We observe 365,636 pixels in the healthy image (566 × 646) and 292,352 pixels in the companion image (571 × 512). Both are numeric matrices with 0 missing values in total. We fit all candidate models exclusively to the healthy pixels; we reserve the tumor image for subsequent cross-image analysis.

3 2. K-Means Model-Selection Framework

3.1 2.1 What we optimize

We fit one-dimensional K-Means to grayscale intensities. For a chosen number of clusters *k*, we minimize the total within-cluster sum of squares:

$$\mathrm{WCSS}(k) = \sum_{j=1}^{k} \sum_{i \in C_j} (x_i - \mu_j)^2,$$

where x_i is the observed intensity of pixel i, C_j is cluster j, and μ_j is its centroid. We calculate WCSS using `sum(model$withinss)` and verify it against the model's `tot.withinss`.

We expect WCSS to decrease (or stay unchanged at the global optimum) as *k* increases because more centroids can represent more intensity levels. We therefore **cannot select k by minimizing WCSS alone**: doing so would systematically favor more clusters. Instead, we inspect the improvement relative to model complexity.

3.2 2.2 Estimation Strategy

We evaluate *k* = 2, 3, …, 10 using the full observed healthy MRI intensity vector. We set a seed and use multiple random starts (`nstart = 5`) to reduce dependence on a single initialization. We keep `iter.max = 1000`. This expands the baseline K-Means specification while retaining the same pixel-level objective and input data.

Computational note. We have 365,636 pixel observations, so fitting multiple models may take time on a laptop. We intentionally do not subsample the image: the reported WCSS values refer to the entire healthy MRI.

Hide

```
k_values <- 2:10
set.seed(1)

models <- lapply(k_values, function(k) {
  kmeans(healthy_vector, centers = k, iter.max = 1000,
         nstart = 5, algorithm = "Hartigan-Wong")
})
names(models) <- as.character(k_values)

stopifnot(all(vapply(models, function(m) {
  isTRUE(all.equal(sum(m$withinss), m$tot.withinss,
                   tolerance = 1e-8)) &&
    sum(m$size) == length(healthy_vector) &&
    all(is.finite(m$centers)) &&
    all(m$size > 0)
}, logical(1))))

wcss <- vapply(models, function(m) m$tot.withinss, numeric(1))
model_summary <- data.frame(
  k = k_values,
  WCSS = as.numeric(wcss),
  Between_SS = vapply(models, function(m) m$betweenss, numeric(1)),
  Explained_Variance = vapply(models, function(m) {
    100 * m$betweenss / m$totss
  }, numeric(1)),
  Iterations = vapply(models, function(m) m$iter, integer(1)),
  Convergence_Code = vapply(models, function(m) m$ifault, integer(1))
)
knitr::kable(model_summary, digits = 4,
             caption = "Full-image K-Means comparison for k = 2–10")
```

Full-image K-Means comparison for k = 2–10

	k	WCSS	Between_SS	Explained_Variance	Iterations	Convergence_Code
2	2	2018.7445	3756.199	65.0430	1	0
3	3	843.7113	4931.232	85.3901	3	0
4	4	467.8845	5307.059	91.8980	4	0
5	5	302.1291	5472.815	94.7683	5	0
6	6	220.7184	5554.225	96.1780	2	0
7	7	170.8341	5604.109	97.0418	2	0
8	8	142.5640	5632.380	97.5313	7	0
9	9	104.7350	5670.208	98.1864	3	0

	k	WCSS	Between_SS	Explained_Variance	Iterations	Convergence_Code
10	10	98.0670	5676.877	98.3019	3	0

Code explanation. We fit nine separate K-Means models and retain their full result objects. `tot.withinss` is the total squared distance from each pixel to its assigned centroid. `betweenss / totss` describes the proportion of intensity variation represented by between-cluster differences; it is an in-sample quantization measure, **not** diagnostic predictive accuracy. `Convergence_Code` is the algorithm's reported status, which we inspect before interpreting solutions.

Output interpretation. We obtain a WCSS of 2,018.745 at $k = 2$ and 98.067 at $k = 10$. The five-cluster model has WCSS 302.1291 and accounts for 94.77% of total pixel-intensity variation. These results measure intensity reconstruction, not whether the cluster boundaries coincide with brain anatomy.

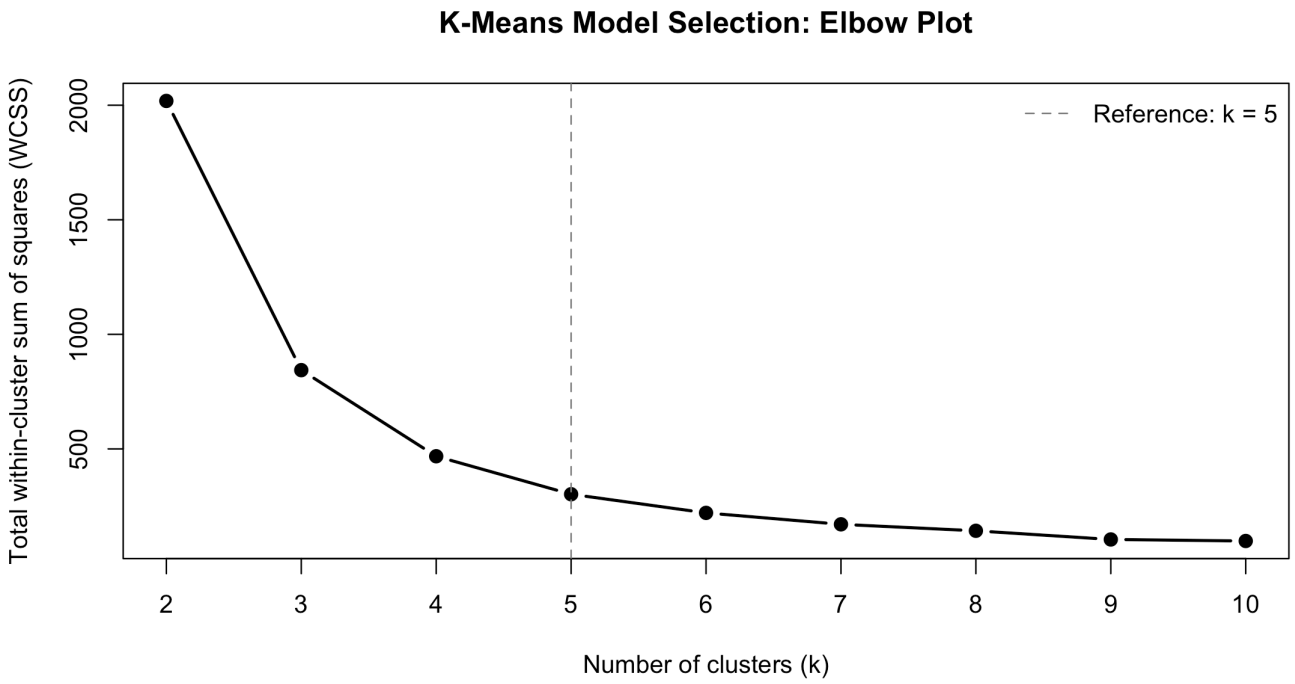
4 3. Elbow Method and Marginal Improvements

4.1 3.1 Visualize the WCSS curve

We plot total WCSS against the number of clusters. We look for a region where the curve begins to flatten: beyond that point, additional clusters provide smaller improvements in within-cluster compactness.

Hide

```
plot(model_summary$k, model_summary$WCSS,
     type = "b", pch = 19, lwd = 2,
     xlab = "Number of clusters (k)",
     ylab = "Total within-cluster sum of squares (WCSS)",
     main = "K-Means Model Selection: Elbow Plot", xaxt = "n")
axis(1, at = k_values)
abline(v = 5, lty = 2, col = "gray50")
legend("topright", legend = "Reference: k = 5",
       lty = 2, col = "gray50", bty = "n")
```



WCSS elbow plot for full-resolution healthy MRI pixels (k = 2–10).

Code explanation. We draw the fitted WCSS values without rescaling or smoothing them. We mark *k* = 5 as a reference, not as an automatically established optimum. We evaluate the visual bend together with the quantitative reductions below.

Output interpretation. We expect WCSS to fall as more centroids become available. A flattening curve suggests diminishing returns, but the location of an elbow can be ambiguous, especially when the underlying grayscale distribution is continuous or contains multiple closely spaced intensity modes. We therefore inspect the size of each additional reduction rather than relying solely on the plot's shape.

4.2 3.2 Quantify gains from each additional cluster

We calculate the absolute decrease in WCSS and the percentage decrease relative to the previous model. The first candidate (*k* = 2) has no preceding model in this grid, so its marginal gain is undefined.

Hide

```
model_summary$WCSS_Reduction <- c(NA_real_, -diff(model_summary$WCSS))
model_summary$Reduction_Percent <- c(
  NA_real_,
  100 * (-diff(model_summary$WCSS)) /
    head(model_summary$WCSS, -1)
)
knitr::kable(model_summary[, c("k", "WCSS", "WCSS_Reduction",
                               "Reduction_Percent")],
             digits = 3,
             caption = "Absolute and percentage reductions in WCSS")
```

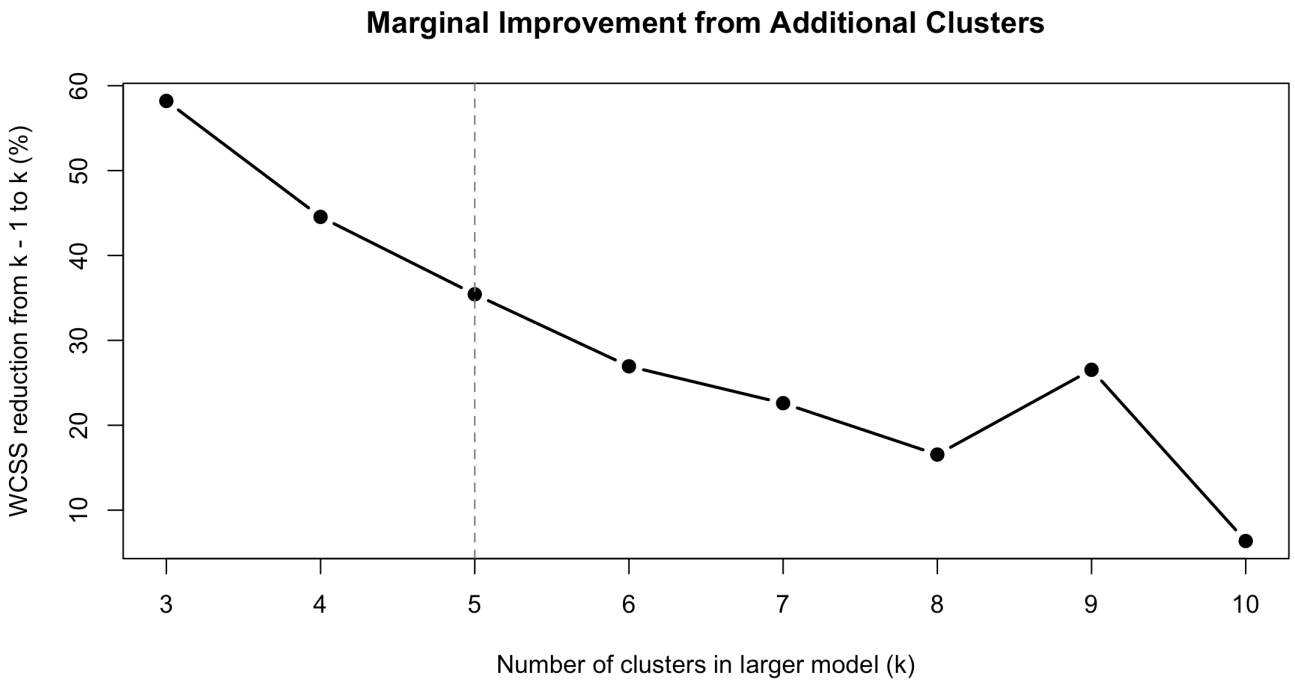
Absolute and percentage reductions in WCSS

	k	WCSS	WCSS_Reduction	Reduction_Percent
2	2	2018.745	NA	NA
3	3	843.711	1175.033	58.206
4	4	467.885	375.827	44.544
5	5	302.129	165.755	35.427
6	6	220.718	81.411	26.946
7	7	170.834	49.884	22.601
8	8	142.564	28.270	16.548
9	9	104.735	37.829	26.535
10	10	98.067	6.668	6.367

Output interpretation. We reduce WCSS by 44.54% when moving from three to four clusters, by 35.43% when moving from four to five, and by 26.95% when moving from five to six. We use these **observed marginal gains** to discuss whether a sixth cluster adds enough intensity detail to justify its complexity. A small percentage reduction is not automatically negligible; its relevance depends on the analytical goal.

Hide

```
plot(model_summary$k[-1], model_summary$Reduction_Percent[-1],
     type = "b", pch = 19, lwd = 2,
     xlab = "Number of clusters in larger model (k)",
     ylab = "WCSS reduction from k - 1 to k (%)",
     main = "Marginal Improvement from Additional Clusters", xaxt = "n")
axis(1, at = k_values[-1])
abline(v = 5, lty = 2, col = "gray50")
```



Percentage reduction in WCSS from adding one cluster.

Code explanation and interpretation. We plot proportional rather than absolute reductions to make successive gains easier to compare. We avoid treating the largest marginal reduction as an automatic selection rule; this plot is a diagnostic for diminishing returns, not a formal statistical test.

5 4. Compare Segmented Images and Cluster Centroids

5.1 4.1 Preserve image orientation

We display the CSV pixel matrix in row-down, column-right image orientation. R's base `image()` uses a different axis convention from conventional image display, so we transpose the matrix and reverse its rows **only for plotting**. We leave all model inputs unchanged.

Hide

```
plot_mri <- function(pixel_matrix, palette, main, zlim = NULL) {
  display_matrix <- t(pixel_matrix[nrow(pixel_matrix):1, , drop = FALSE])
  if (is.null(zlim)) zlim <- range(display_matrix)
  image(display_matrix, axes = FALSE, asp = 1, useRaster = TRUE,
        col = palette, zlim = zlim, main = main,
        xlab = "", ylab = "")
}

# We use a fixed set of colors indexed by intensity rank, not arbitrary
# K-Means cluster ID, so visual comparisons are consistent across k.
segmented_intensity <- function(model, image_matrix) {
  assigned_intensity <- as.numeric(model$centers[model$cluster, 1])
  matrix(assigned_intensity, nrow = nrow(image_matrix),
         ncol = ncol(image_matrix))
}
```

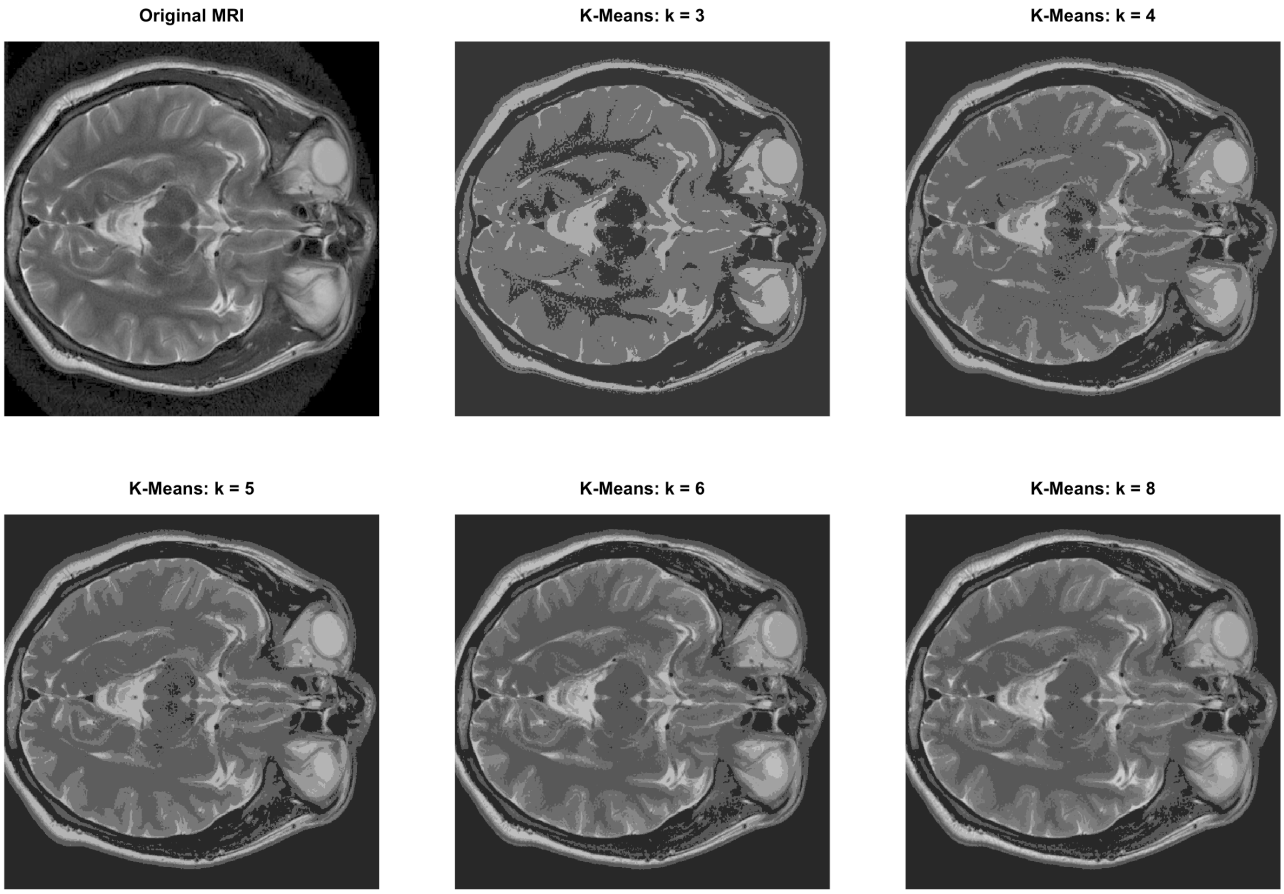
Code explanation. We reconstruct an image by replacing each pixel with its assigned cluster centroid. We use a fixed grayscale palette so that darker and lighter clusters remain comparable across different values of *k*. Raw cluster IDs are arbitrary and cannot be compared numerically between models.

5.2 4.2 Compare k = 3, 4, 5, 6, and 8

We select several representative candidate counts to illustrate the trade-off between coarse segmentation and finer intensity resolution. The grayscale plots represent **centroid intensities**, not ground-truth tissue classes.

Hide

```
selected_k <- c(3L, 4L, 5L, 6L, 8L)
old_par <- par(mfrow = c(2, 3), mar = c(2.5, 2.5, 3, 1))
plot_mri(healthy_matrix, gray.colors(256, start = 0, end = 1),
         "Original MRI", zlim = c(0, 1))
for (k in selected_k) {
  reconstructed <- segmented_intensity(models[[as.character(k)]],
                                       healthy_matrix)
  plot_mri(reconstructed, gray.colors(256, start = 0, end = 1),
           paste("K-Means: k =", k), zlim = c(0, 1))
}
```



Original healthy MRI and centroid-reconstructed segmentations at selected k values.

Hide

```
par(old_par)
```

Output interpretation. We compare how much grayscale detail remains as we increase *k*. At lower *k*, we merge more pixel intensities into a few broad bands. At higher *k*, we represent more intensity distinctions and may reveal finer-looking boundaries. However, a visually detailed segmentation does not establish more accurate anatomy: intensity-only clustering ignores pixel adjacency, image artifacts, and tissue-specific ground truth.

5.3 4.3 Inspect centroid positions and pixel proportions

We order cluster centroids by intensity because K-Means cluster numbers have no inherent meaning. We then calculate the fraction of pixels assigned to each ordered centroid.

Hide

```
        centroid_profile <- function(model, k) {
          ordering <- order(as.numeric(model$centers[, 1]))
          data.frame(
            k = k,
            Intensity_Rank = seq_len(k),
            Centroid = as.numeric(model$centers[ordering, 1]),
            Pixels = as.integer(model$size[ordering]),
            Percent = 100 * as.numeric(model$size[ordering]) /
              sum(model$size)
          )
        }
        profiles <- do.call(rbind, lapply(c(4L, 5L, 6L), function(k) {
          centroid_profile(models[[as.character(k)]], k)
        }))
        rownames(profiles) <- NULL
        knitr::kable(profiles, digits = 4,
                     caption = "Intensity-ranked centroids and cluster proportion
s for k = 4, 5, and 6")
```

Intensity-ranked centroids and cluster proportions for k = 4, 5, and 6

k	Intensity_Rank	Centroid	Pixels	Percent
4	1	0.0244	147076	40.2247
4	2	0.1321	138478	37.8732
4	3	0.2542	53823	14.7204
4	4	0.4587	26259	7.1817
5	1	0.0219	139963	38.2793
5	2	0.1166	114480	31.3098
5	3	0.2014	65557	17.9296
5	4	0.3299	27338	7.4768
5	5	0.4914	18298	5.0044
6	1	0.0196	133162	36.4193
6	2	0.1024	90451	24.7380
6	3	0.1668	71186	19.4691
6	4	0.2501	36521	9.9883
6	5	0.3796	22423	6.1326
6	6	0.5236	11893	3.2527

Code explanation. We use `order()` to sort centroids from darkest to lightest. We reorder the corresponding cluster sizes in the same way. This allows us to compare intensity bands across models without confusing arbitrary numeric cluster labels with stable tissue identities.

Output interpretation. We interpret each centroid as a representative grayscale level, and each proportion as the share of all image pixels assigned to that level. These proportions include any background pixels present in the CSV. We do not label the clusters as specific tissues because the data provide no anatomical annotations.

6 5. Sensitivity to Random Initialization

6.1 5.1 Why initialization matters

Here we use K-Means, an iterative algorithm that can converge to different local solutions depending on the initial centroids. To examine this sensitivity, we fit multiple single-start models at $k = 4$ and $k = 5$. We compare their WCSS values and intensity-ranked centroids. We use a fixed sequence of seeds to make the exercise reproducible.

We deliberately keep this experiment separate from the multi-start models in Section 2: the single-start runs reveal how much initialization matters, while `nstart = 5` provides a stronger practical candidate fit.

Hide

```
seeds <- 1:10
stability_models <- list()
stability_rows <- list()
for (k in c(4L, 5L)) {
  for (seed in seeds) {
    set.seed(seed)
    fit <- kmeans(healthy_vector, centers = k, iter.max = 1000,
                  nstart = 1, algorithm = "Hartigan-Wong")
    key <- paste(k, seed, sep = "_")
    stability_models[[key]] <- fit
    stability_rows[[key]] <- data.frame(
      k = k, Seed = seed, WCSS = fit$tot.withinss,
      Convergence_Code = fit$ifault
    )
  }
}
stability_results <- do.call(rbind, stability_rows)
rownames(stability_results) <- NULL
stability_summary <- do.call(rbind, lapply(split(stability_results,
                                                 stability_results$k),
                                           function(d) {
  data.frame(k = d$k[1], Runs = nrow(d),
             Minimum_WCSS = min(d$WCSS),
             Median_WCSS = median(d$WCSS),
             Maximum_WCSS = max(d$WCSS),
             Range_WCSS = diff(range(d$WCSS)),
             Relative_Range_Percent = 100 * diff(range(d$WCSS)) /
               min(d$WCSS))
}))
rownames(stability_summary) <- NULL
knitr::kable(stability_summary, digits = 5,
             caption = "Initialization sensitivity over ten single-start
runs")
```

Initialization sensitivity over ten single-start runs

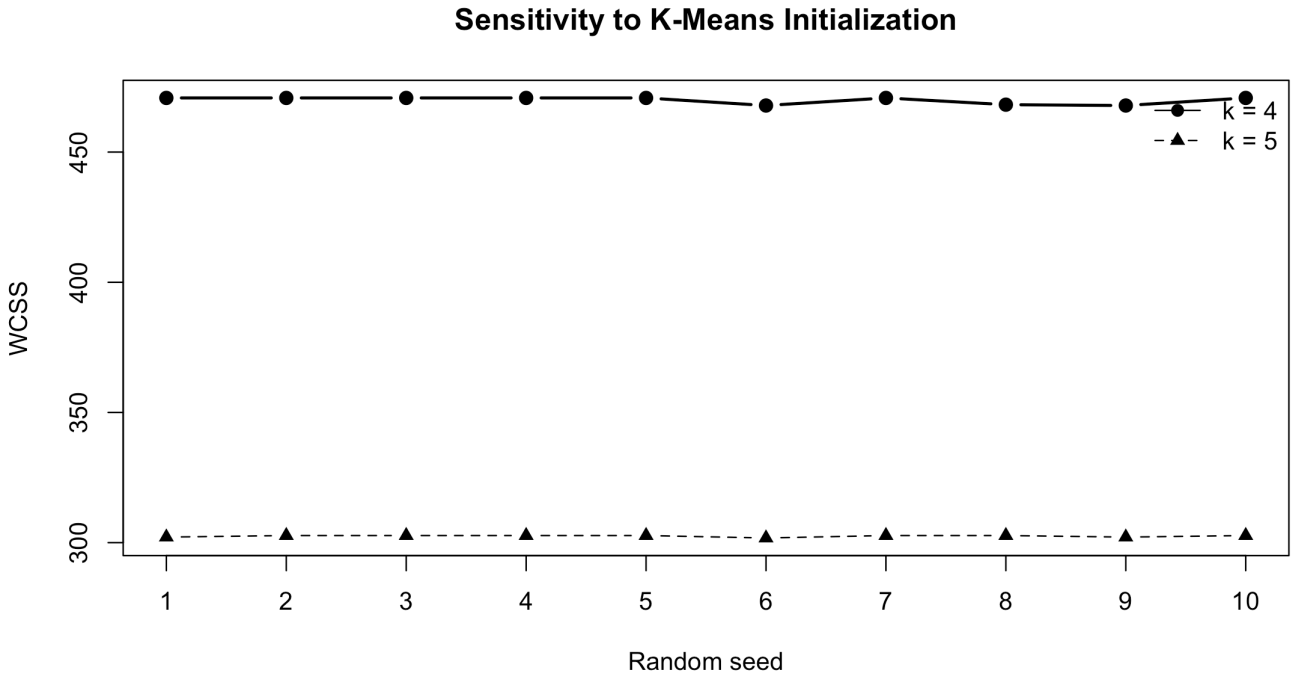
k	Runs	Minimum_WCSS	Median_WCSS	Maximum_WCSS	Range_WCSS	Relative_Range_Percent
4	10	467.8845	470.7996	470.7996	2.91509	0.62304
5	10	301.7859	302.7250	302.7250	0.93908	0.31117

Code explanation. We refit each candidate with seeds 1–10, one initialization per seed. We summarize the minimum, median, maximum, and spread of WCSS. The relative range scales the spread by the best observed single-start WCSS. This is a practical sensitivity check, not a formal confidence interval or statistical significance test.

Output interpretation. We observe a relative WCSS range of 0.623% for *k* = 4 and 0.3112% for *k* = 5 across the tested seeds. A smaller spread indicates more consistent objective values under these initializations; it does not prove that all runs recover the global optimum. We also inspect the convergence codes reported by R.

Hide

```
plot(seeds, stability_results$WCSS[stability_results$k == 4],
     type = "b", pch = 19, lwd = 2,
     xlab = "Random seed", ylab = "WCSS",
     main = "Sensitivity to K-Means Initialization",
     ylim = range(stability_results$WCSS), xaxt = "n")
axis(1, at = seeds)
lines(seeds, stability_results$WCSS[stability_results$k == 5],
      type = "b", pch = 17, lty = 2)
legend("topright", legend = c("k = 4", "k = 5"),
       pch = c(19, 17), lty = c(1, 2), bty = "n")
```



Single-start WCSS values for k = 4 and k = 5 across ten fixed seeds.

Output interpretation. We use this plot to compare within-*k* variation across random starts, not to compare the absolute WCSS levels as if they were normalized for model complexity. The lower WCSS of a higher-*k* model is expected and is not by itself evidence of a better scientific segmentation.

7 6. Model Selection and Interpretation

7.1 6.1 Synthesize fit, complexity, and stability

We combine the WCSS curve, the marginal improvement table, the reconstructed MRI images, and the random-start sensitivity results. We do not select *k* by a single numerical threshold, because no ground-truth segmentation is available to define an objectively correct number of tissue regions.

We retain **k = 5 as the reference specification** for the subsequent cross-image analysis. We also retain **k = 4 as a comparison model**, since it offers a more parsimonious intensity partition. We treat this as an interpretable analytical choice, rather than a proof that either model corresponds to a medically correct number of tissue classes.

Hide

```
        selected_summary <- model_summary[model_summary$k %in% c(4L, 5L, 6L),
                                          c("k", "WCSS", "Explained_Variance",
                                            "Reduction_Percent", "Convergence_Cod
e")]
        knitr::kable(selected_summary, digits = 4,
                     caption = "Decision-relevant model diagnostics near k = 5")
```

Decision-relevant model diagnostics near k = 5

	k	WCSS	Explained_Variance	Reduction_Percent	Convergence_Code
4	4	467.8845	91.8980	44.5445	0
5	5	302.1291	94.7683	35.4266	0
6	6	220.7184	96.1780	26.9457	0

Output interpretation. We use the three neighboring models to inspect the additional WCSS reduction obtained by increasing *k*. We compare the changes with the segmentation images and the stability results before choosing how much grayscale detail to preserve. Because this is an unsupervised, one-feature problem, we do not interpret explained variance as a classification accuracy metric.

7.2 6.2 Important limitations

- **We use intensity only.** We do not model neighboring pixels, edges, spatial smoothness, or image texture. Disconnected pixels can share the same cluster.
- **We lack annotated segmentation masks.** We cannot calculate Dice, intersection-over-union, sensitivity, or specificity for anatomical or tumor regions.
- **We analyze one training image.** We cannot infer patient-level generalizability or clinical robustness from a single healthy MRI.
- **We include the image background.** We fit the full CSV without a validated brain mask; background intensities can influence centroid placement and cluster proportions.
- **We inspect in-sample fit.** We do not interpret the elbow or WCSS improvements as out-of-sample prediction quality.

- **We cannot diagnose tumors.** We use the tumor image only as a later test of transferring intensity clusters; its clinical content is not labeled in the source files.

8 7. Conclusion

We evaluate K-Means segmentation of 365,636 observed healthy-MRI pixels across nine candidate cluster counts, from two to ten. We quantify the WCSS–complexity trade-off, compare segmented images and ordered centroids, and assess sensitivity to random initialization. We retain five clusters as an interpretable reference while keeping four clusters as a parsimonious alternative.

Our results establish how different values of *k* represent grayscale intensity variation in this image. They do **not** establish clinically meaningful tissue boundaries or tumor-detection performance. We next apply the learned healthy-image intensity centroids to the separate MRI in `tumor.csv` , without treating cross-image cluster assignment as supervised diagnosis.

9 8. Reproducibility

Download and read the two original CSVs and follow the code to reproduce the results.

Hide

```
sessionInfo()
```

```
## R version 4.2.1 (2022-06-23)
## Platform: x86_64-apple-darwin17.0 (64-bit)
## Running under: macOS Big Sur ... 10.16
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.2/Resources/lib/libRblas.0.
dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.2/Resources/lib/libRlapack.
dylib
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.33     R6_2.5.1          lifecycle_1.0.4   jsonlite_1.8.8
##  [5] evaluate_0.23     highr_0.10        cachem_1.0.8      rlang_1.1.2
##  [9] cli_3.6.2         rstudioapi_0.15.0 jquerylib_0.1.4   bslib_0.6.1
## [13] rmarkdown_2.25    tools_4.2.1       xfun_0.41         yaml_2.3.8
## [17] fastmap_1.1.1     compiler_4.2.1    htmltools_0.5.7   knitr_1.45
## [21] sass_0.4.8
```

10 References

- MIT OpenCourseWare. *15.071 The Analytics Edge*, Unit 6 Recitation: *Seeing the Big Picture: Segmenting Images to Create Data* (`Unit6_Recitation.pdf`).
- Course analysis code: `Unit6_Recitation.R` .
- K-Means model-selection and cross-image analysis code: `week6_3.2.R` .
- Project data: `healthy.csv` and `tumor.csv` (grayscale MRI pixel matrices supplied with the course materials).